



Announcements

- The lecture will stay on Wednesdays, 16:00 - 17:30.
- Tomorrow (24.10.) is *Dies Academicus* ('Feierliche Immatrikulation') → no exercise session!

Announcements

- The lecture will stay on Wednesdays, 16:00 - 17:30.
- Tomorrow (24.10.) is *Dies Academicus* ('Feierliche Immatrikulation') → no exercise session!
- Is anybody here still missing **git** access?

Announcements

- The lecture will stay on Wednesdays, 16:00 - 17:30.
- Tomorrow (24.10.) is *Dies Academicus* ('Feierliche Immatrikulation') → no exercise session!
- Is anybody here still missing **git** access?
- About code in the oral exam: We will present code with our questions where required.

General Training for Programming Competitions & Coding Interviews

October 30, 2024

Bit Manipulation, Primitive Types, and Arrays

Markus Fischer, Paul G. Rump, Christoph Staudt

Faculty of Mathematics and Computer Science



Bit manipulation

Bit Manipulation Basics

Bitwise operations are among the **most basic operations** in programming. They allow us to manipulate individual bits in binary data.

bit_manipulation_basics_secret.cpp 

```
1  uint8_t number1 = 0b00000110;
2  uint8_t number2 = 0b00001010;
3  // these operations occur bit-by-bit, with what's happening
4  // on one bit position never impacting the other bits
5  uint8_t result_A = number1 | number2;           // 00001110
6  uint8_t result_B = number1 & number2;           // 00000010
7  uint8_t result_C = number1 ^ number2;           // 00001100
8  uint8_t result_D = ~number1;                    // 11111001
9  // here we actually "overwrite" bits
10 uint8_t result_E = number1 << 2u;                // 00011000
11 uint8_t result_F = number1 >> 2u;                // 00000001
12 // common mathematical operations also "overwrite" bits
13 uint8_t result_addition = number1 + number2;     // 00010000
```

Bit Manipulation Basics

Bitwise operations are among the **most basic operations** in programming. They allow us to manipulate individual bits in binary data.

bit_manipulation_basics.cpp ➡

```
1  uint8_t number1 = 0b00000110;
2  uint8_t number2 = 0b00001010;
3  // these operations occur bit-by-bit, with what's happening
4  // on one bit position never impacting the other bits
5  uint8_t result_or = number1 | number2;      // 00001110
6  uint8_t result_and = number1 & number2;     // 00000010
7  uint8_t result_xor = number1 ^ number2;     // 00001100
8  uint8_t result_not = ~number1;              // 11111001
9  // here we actually "overwrite" bits
10 uint8_t result_left_shift = number1 << 2u;  // 00011000
11 uint8_t result_right_shift = number1 >> 2u; // 00000001
12 // common mathematical operations also "overwrite" bits
13 uint8_t result_addition = number1 + number2; // 00010000
```


Smart Bit Manipulation Can Speed-Up Your Code

Programming tasks that need bit manipulation include low-level device control, error detection and correction algorithms, data compression and encryption algorithms, but they can also be used for **optimization**.

In some cases, bit manipulation **can reduce the need to loop** over a data structure and **can give many-fold speedups**, as bit manipulations are processed at the same time.

Smart Bit Manipulation Can Speed-Up Your Code

no_loop.cpp 

```
1  // imagine we represent a set as an array of booleans
2  bool set1[8] = {true, false, true, false, false, true, false, false};
3  bool set2[8] = {false, true, true, false, false, false, false, true};
4
5  // test if there are common elements
6  bool have_common_entries = false;
7  for (int i = 0; i < 8; ++i)
8      if (set1[i] & set2[i])
9          have_common_entries = true;
10
11 // a set can also be represented by a number
12 uint8_t set1_num = 0b10100100;
13 uint8_t set2_num = 0b01100001;
14
15 // test if there are common bits
16 bool have_common_bits = set1_num & set2_num; // if not 0 than true
```

Representing Sets

Naive implementation of a set: an array (or `std::vector`) that contains the elements of the set.

Problem: all set operations **require a loop** over the data structure. For example:

- Check if an item is contained in the set
- Merge two sets
- Intersect two sets

Representing Sets

Every subset of a set $\{0, 1, 2, \dots, n - 1\}$ can be represented as an n bit integer whose one bits indicate which elements belong to the subset.

Implementing **set operations** as **bit operations**

Operation	Set syntax	Bit syntax
Intersection	$a \cap b$	$a \& b$
Union	$a \cup b$	$a b$
Complement	$\bar{a}$	$\sim a$
Difference	$a \setminus b$	$a \& (\sim b)$

There is also a `std::bitset` that makes working with bits quite convenient.

<https://www.geeksforgeeks.org/c-bitset-and-its-application/>

The Sign Bit

To be able to represent negative numbers, most datatypes have one specific bit reserved to encode the sign. This is the **sign bit**, which is usually the **most significant bit** ("left-most bit").

When using bit manipulations for arithmetic operations, it is important to consider the sign bit and the encoding of negative numbers.

Negative Integers

The binary representation of $-K$ (negative K) as an N-bit number is *concatenate*(**1**, $2^{N-1} - K$). The **1** is the **sign bit** to indicate that the number is **negative**.

To make $-K$ we could also **invert** the **bits in K** and **add 1**.

negative_numbers.cpp 

```
1 signed char a = 3;           // a
2 signed char b = -a;          // -a
3 signed char c = (128 - a) ^ 0b10000000; // -a
4 signed char d = ~a + 1;       // -a
```

Arithmetic vs. Logical Right Shift

There are **two types** of **right shifts** for **negative signed integers** (implementation and/or platform dependent):

- **Logical right shift** → we shift the bits and **put a 0 in the most significant bit**
- **Arithmetic right shift** → we shift values to the right but fill in the **new bits** with the **value of the sign bit** (this has the effect of 'roughly' dividing by two)

arithmetic_vs_logical_right_shift.cpp 

```
1 char a = -30;           // a = -30 --> 11100010
2 // probably arithmetic right shift
3 char b = a >> 2;        // b = -8  --> 11111000
4 // cast char to unsigned to perform a logical right shift
5 char c = ((unsigned char &a) >> 2; // c = 56 --> 00111000
```

Arithmetic vs. Logical Right Shift

There are **two types** of **right shifts** for **negative signed integers** (implementation and/or platform dependent):

- **Logical right shift** → we shift the bits and **put a 0 in the most significant bit**
- **Arithmetic right shift** → we shift values to the right but fill in the **new bits** with the **value of the sign bit** (this has the effect of 'roughly' dividing by two)

arithmetic_vs_logical_right_shift.cpp 

```
1 char a = -30;           // a = -30 --> 11100010
2 // probably arithmetic right shift
3 char b = a >> 2;        // b = -8  --> 11111000
4 // cast char to unsigned to perform a logical right shift
5 char c = ((unsigned char &a) >> 2; // c = 56 --> 00111000
```



Use **unsigned integers** to **avoid shifting errors**.

Sorting Floats With an Integer Routine

The IEEE format was designed in such a way that **floats can be sorted** with **integer sorting routines**.

Casting integers to floats keeps the relative absolute value: **large float numbers become large int numbers** and smaller float numbers become smaller int numbers.

But we need to invert the bits of negative numbers:

```
1 inline void invert_bits(int x){  
2     x = x^((x>>31) & 0x7FFFFFFF);  
3 }
```

Sorting Floats With an Integer Routine

The IEEE format was designed in such a way that **floats can be sorted** with **integer sorting routines**.

Casting integers to floats keeps the relative absolute value: **large float numbers become large int numbers** and smaller float numbers become smaller int numbers.

But we need to invert the bits of negative numbers:

```
1 inline void invert_bits(int x){  
2     x = x^((x>>31) & 0x7FFFFFFF);  
3 }
```

If the sign bit is zero, this code does nothing, but if the sign bit is 1, all bits but the sign bit are inverted.

Sorting Floats With an Integer Routine

Using this, we can transform an array of floats to an array of integers.

sort_floats_fast.cpp (sort_floats_fast.tar.gz) 

```
1 // call this before and after sorting with a
2 // sorting routine for int
3 inline void convert_float(float *arr, int n) {
4     for (int i = 0; i < n; ++i) {
5         int &x = (int &)arr[i];
6         x = x ^ ((x >> 31) & 0x7FFFFFFF);
7     }
8 }
```



If you need a **fast sorting routine** for **floats**, you can **use one for integers**.

Common Bit Tasks

Task A

common_bit_tasks_secret.cpp 

```
1 // ???  
2 inline bool bit_op_a(uint32_t num, uint32_t i) {  
3     return num & (1u << i);  
4 }
```

Task B

common_bit_tasks_secret.cpp 

```
1 // ???  
2 inline uint32_t bit_op_b(uint32_t num, uint32_t i) {  
3     return num | (1u << i);  
4 }
```

Common Bit Tasks

Task A

common_bit_tasks.cpp 

```
1 // get bit at position i
2 inline bool get_bit(uint32_t num, uint32_t i) {
3     return num & (1u << i); // implicit conversion to bool
4 }
```

Task B

common_bit_tasks_secret.cpp 

```
1 // ???
2 inline uint32_t bit_op_b(uint32_t num, uint32_t i) {
3     return num | (1u << i);
4 }
```

Common Bit Tasks

Task A

common_bit_tasks.cpp 

```
1 // get bit at position i
2 inline bool get_bit(uint32_t num, uint32_t i) {
3     return num & (1u << i); // implicit conversion to bool
4 }
```

Task B

common_bit_tasks.cpp 

```
1 // sets the bit at position i to one
2 inline uint32_t set_bit(uint32_t num, uint32_t i) {
3     return num | (1u << i);
4 }
```

Common Bit Tasks

Task C

common_bit_tasks_secret.cpp 

```
1 // ???  
2 inline uint32_t bit_op_c(uint32_t num, uint32_t i) {  
3     return num & ~(1u << i);  
4 }
```

Task D

common_bit_tasks_secret.cpp 

```
1 // ???  
2 inline uint32_t bit_op_d(uint32_t num, uint32_t i) {  
3     return num & ((1u << i) - 1);  
4 }
```

Common Bit Tasks

Task C

common_bit_tasks.cpp 

```
1 // clears the bit at position i to zero
2 inline uint32_t clear_bit(uint32_t num, uint32_t i) {
3     return num & ~(1u << i);
4 }
```

Task D

common_bit_tasks_secret.cpp 

```
1 // ???
2 inline uint32_t bit_op_d(uint32_t num, uint32_t i) {
3     return num & ((1u << i) - 1);
4 }
```


Common Bit Tasks

Task C

common_bit_tasks.cpp 

```
1 // clears the bit at position i to zero
2 inline uint32_t clear_bit(uint32_t num, uint32_t i) {
3     return num & ~(1u << i);
4 }
```

Task D

common_bit_tasks.cpp 

```
1 // clear all bits from the most significant bit through i (inclusive)
2 inline uint32_t clear_bits_msb_through_i(uint32_t num, uint32_t i) {
3     return num & ((1u << i) - 1);
4 }
```

Common Bit Tasks

Task E

common_bit_tasks_secret.cpp 

```
1 // ???  
2 inline uint32_t bit_op_e(uint32_t num, uint32_t i) {  
3     return num & (UINT32_MAX << (i + 1));  
4 }
```

Common Bit Tasks

Task E

common_bit_tasks.cpp 

```
1 // clear all bits from i through 0 (inclusive)
2 inline uint32_t clear_bits_i_through_0(uint32_t num, uint32_t i) {
3     return num & (UINT32_MAX << (i + 1)); // UINT32_MAX contains only 1s
4 }
```

Bit Manipulation Task F

`(n & (n-1)) == 0`

Bit Manipulation Task F

$$(n \ \& \ (n-1)) == 0$$

Let's take it step by step:

- What do the bits of $n-1$ look like? Example: $n = 0b01010100$

Bit Manipulation Task F

$$(n \ \& \ (n-1)) == 0$$

Let's take it step by step:

- What do the bits of $n-1$ look like? Example: $n = 0b01010100$
→ Every bit starting with the last 1 is flipped. Example: $(n-1) = 0b01010011$

Bit Manipulation Task F

$$(n \ \& \ (n-1)) \ == \ 0$$

Let's take it step by step:

- What do the bits of $n-1$ look like? Example: $n = 0b01010100$
- Every bit starting with the last 1 is flipped. Example: $(n-1) = 0b01010011$
- What do the bits of $(n \ \& \ (n-1))$ look like?

Bit Manipulation Task F

$$(n \& (n-1)) == 0$$

Let's take it step by step:

- What do the bits of $n-1$ look like? Example: $n = 0b01010100$
→ Every bit starting with the last 1 is flipped. Example: $(n-1) = 0b01010011$
- What do the bits of $(n \& (n-1))$ look like?
→ Like n , but the last 1 (if any) is removed. Example: $(n \& (n-1)) = 0b01010000$

Bit Manipulation Task F

$$(n \ \& \ (n-1)) \ == \ 0$$

Let's take it step by step:

- What do the bits of $n-1$ look like? Example: $n = 0b01010100$
→ Every bit starting with the last 1 is flipped. Example: $(n-1) = 0b01010011$
- What do the bits of $(n \ \& \ (n-1))$ look like?
→ Like n , but the last 1 (if any) is removed. Example: $(n \ \& \ (n-1)) = 0b01010000$
- When is the result 0 (meaning $0b00000000$)?

Bit Manipulation Task F

$$(n \ \& \ (n-1)) \ == \ 0$$

Let's take it step by step:

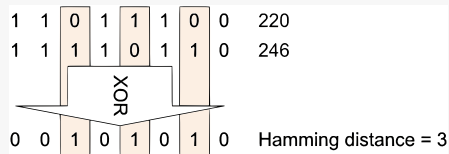
- What do the bits of $n-1$ look like? Example: $n = 0b01010100$
→ Every bit starting with the last 1 is flipped. Example: $(n-1) = 0b01010011$
- What do the bits of $(n \ \& \ (n-1))$ look like?
→ Like n , but the last 1 (if any) is removed. Example: $(n \ \& \ (n-1)) = 0b01010000$
- When is the result 0 (meaning $0b00000000$)?
→ If there was at most a single 1-bit in n !



The expression checks for powers of 2! (Or $n = 0$).

Binary Hamming Distance

Write a function to determine the **number of bits** you would need to **flip** to **convert** integer **A** to integer **B**.



Binary Hamming Distance

hamming.cpp 

```
1 // computing hamming distance by clearing the least significant 1
2 // in n = a ^ b with n & (n - 1) and counting until n reaches 0
3 inline uint32_t hamming(uint32_t a, uint32_t b) {
4     uint32_t count = 0;
5     for (uint32_t n = a ^ b; n != 0; n = n & (n - 1)) {
6         count++;
7     }
8     return count;
9 }
10
11 // simpler: hamming distance can also be computed with bitsets
12 inline uint32_t hamming_bitset(bitset<32> a, bitset<32> b) {
13     return (a ^ b).count();
14 }
```

Pairwise Swap

Write a program to **swap odd and even bits** in an unsigned integer with as **few instructions as possible** (e.g., bit 0 and bit 1 are swapped, bit 2 and bit 3 are swapped, and so on).



Pairwise Swap

Write a program to **swap odd and even bits** in an unsigned integer with as **few instructions as possible** (e.g., bit 0 and bit 1 are swapped, bit 2 and bit 3 are swapped, and so on).

pairwise_swap.cpp 

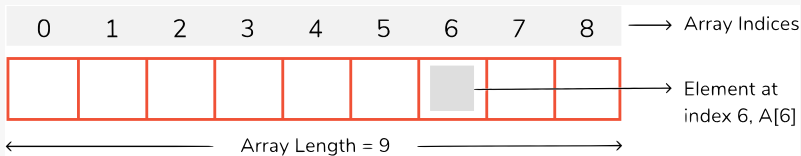
```
1 inline uint32_t pairwise_swap(uint32_t num) {  
2     // hex(a) -> bin(1010), hex(5) -> bin(0101)  
3     return ((num & 0xaaaaaaaau) >> 1u) | ((num & 0x55555555u) << 1u);  
4 }
```

Other **amazing / useful bit hacks**:

<https://graphics.stanford.edu/~seander/bithacks.html>

Arrays

Arrays



- Array problems often have **simple brute-force solutions** that use $\mathcal{O}(n)$ space, but **subtler solutions** that use the array itself to reduce space complexity to $\mathcal{O}(1)$.
- **Instead of deleting** an entry (which requires moving subsequent entries), consider **overwriting** it.
- It's incredibly **easy to make off-by-1 errors** when operating on arrays.
- **Don't worry** about **preserving** the **integrity** of the array (sortedness, keeping equal entries together, etc.) **until** it is **time to return**.

Example Interview Problem (Arrays)

Your input is an array of unsigned 64 bit integers, and you have to **reorder** its **entries** so that the **odd entries appear first**. (Requirements: $\mathcal{O}(1)$ space, $\mathcal{O}(n)$ time)



Example Interview Problem (Arrays)

Your input is an array of unsigned 64 bit integers, and you have to **reorder** its **entries** so that the **odd entries appear first**. (Requirements: $\mathcal{O}(1)$ space, $\mathcal{O}(n)$ time)

reorder.cpp 

```
1 inline void reorder(uint64_t *array, size_t n) {  
2     size_t left = 0;  
3     for (size_t i = 0; i < n; ++i) {  
4         if (array[i] % 2) { // if odd  
5             swap(array[i], array[left++]);  
6         }  
7     }  
8 }
```

But can it be faster?

Delete Duplicates From a Sorted Array

Write a program which takes as input a **sorted array** and **updates it** so that all **duplicates** have been **removed** and the **remaining elements** have been **shifted left** to fill the emptied indices. Return the **number of valid elements**. **Library functions** for performing this operation are **not allowed**. (Requirements: $\mathcal{O}(1)$ space, $\mathcal{O}(n)$ time)

Draw an example: Given array $\langle 2, 3, 5, 5, 7, 11, 11, 11, 13 \rangle$, then after deletion, the array is $\langle 2, 3, 5, 7, 11, 13, 0, 0, 0 \rangle$. After deleting repeated elements, there are 6 valid entries. There are no requirements as to the values stored beyond the last valid element.

Deleting Duplicates From a Sorted Array in $\mathcal{O}(n)$ Time and $\mathcal{O}(1)$ Space

remove_duplicates.cpp 

```
1 inline size_t delete_duplicates(vector<int> &v) {
2     if (v.empty()) return 0;
3     size_t write_idx = 1;
4     for (size_t i = 1; i < v.size(); ++i) {
5         // if integer under v[i] appears for the first time
6         if (v[i] != v[write_idx - 1]) {
7             v[write_idx++] = v[i];
8         }
9     }
10    return write_idx;
11 }
```

< 2, 3, 5, 5, 7, 11, 11, 11, 13 > $\rightarrow$ **< 2, 3, 5, 7, 11, 13, 11, 11, 13 >**

Exercises (Upload Solutions to your Repository)

1. Given a vector of **fruits baskets** and a vector of **fruits preferences** of different people, compute for each person the **number of fruits baskets with at least 3 fruits** of that person's preferences. 

`fruits_baskets.tar.gz` 

2. **Reverse bits** with a **lookup table**.

`reverse_bits_with_lookup.cpp` 

3. Find the **length** of the **longest sequence** of 1s you can create by **flipping** exactly **one bit** in an integer from a 0 to a 1.

`max_flip_sequence.cpp` 

4. **Remove** all occurrences of a specific **value** in a **sorted array**.

`delete_keys.cpp` 

Exam Questions

- **How** can we **exploit bit operations** for operations with **sets**?
- Explain the **difference** between **arithmetic** and **logical right shifts** for **negative signed integers**.
- Outline an **algorithm** to **count** the number of **bits** that are **set** to 1 in an integer. (no bitset or built-in functions)
- Outline an **algorithm** to **reverse** the bits in an unsigned integer (swap bits at position i and $n - i$).